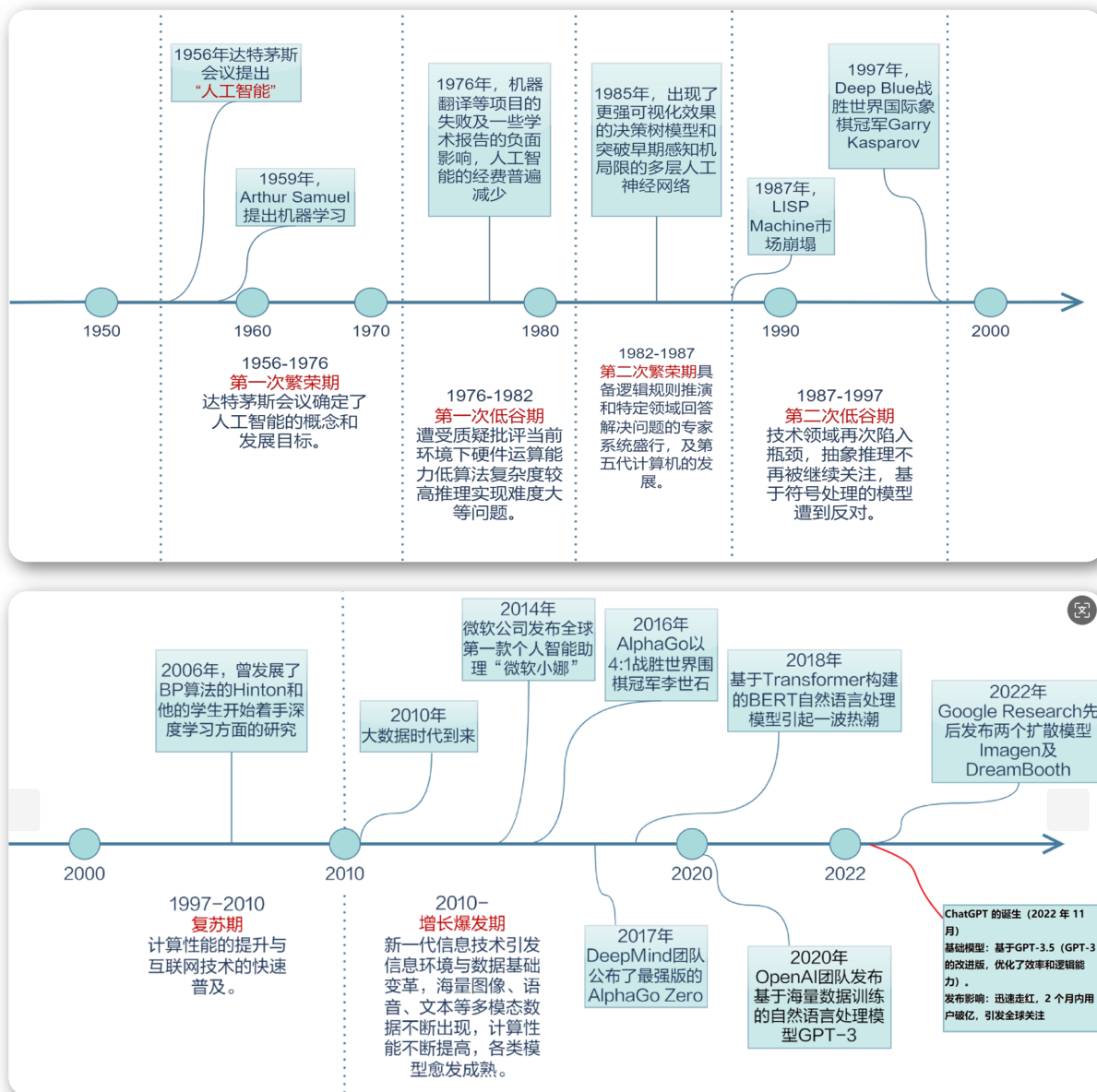


AI的发展历程

AI是人工智能（Artificial Intelligence）的缩写，它是一种模拟人类智能的技术，使机器能够像人一样学习、思考和做出决策，从而能够自主地执行各种任务。AI的核心目标是让机器能够执行通常需要人类智能的任务，例如语言理解、图像识别、复杂问题解决等。



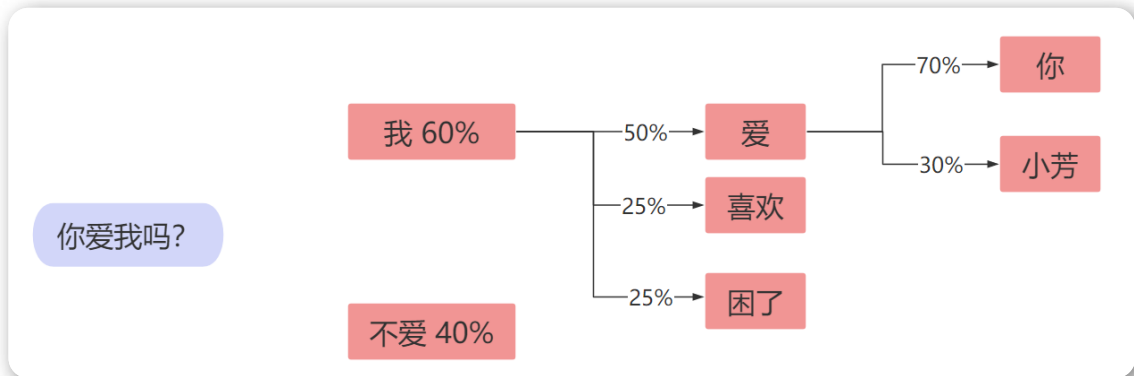
1 规则驱动阶段 (1950s-1980s)

基于规则的人工智能，这一阶段，AI以规则为基础的专家系统，依赖预设的逻辑和规则。

- 图灵测试（1950）：阿兰·图灵提出衡量机器是否具有智能的方法：图灵测试。这是人工智能最早的概念基础之一。（如果一个人通过与机器和另一个人分别交流，无法分辨出哪一方是机器，那么就认为这台机器通过了图灵测试，表现出了类似人类的智能。）

2 机器学习阶段（1990s-2010s初）

通过数据训练模型，使机器能够从数据中学习规律。



3 深度学习阶段（2013s-2018）

随着深度学习的兴起，AI进入了以神经网络为核心的方法阶段。在这一阶段，科学家利用神经网络结构，模仿人脑的复杂结构来解释数据，例如图像，声音和文本，处理更复杂的任务。

4 大模型阶段（2018至今）

这一阶段以**大规模数据**和**算力**为基础，构建**通用性强、性能卓越**的AI模型。目前我们使用的AI产品几乎都是大模型产品，如DeepSeek, GPT, Qwen等等。

实时效果反馈

1. 如果一个人通过与机器和另一个人分别交流，无法分辨出哪一方是机器，那么就认为

A 这台机器通过了图灵测试

- ☐ B 这台机器没有通过图灵测试
- ☐ C 无法确定这台机器是否通过图灵测试
- ☐ D 以上都不正确

2. 以下模型属于大模型的是

- ☐ A DeepSeek
- ☐ B GPT
- ☐ C Qwen
- ☐ D 以上都正确

答案

1=>A 2=>D

常见AI产品的类型



我们可以将市面上常见AI分为两大类：

- **分析式AI**：也称为判别式AI，其核心任务是对已有数据进行分类、预测或决策。优势在于其高精度和高效性，但其局限性在于仅能处理已有数据的模式，无法创造新内容。
- **生成式AI**：专注于创造新内容，例如文本、图像、音频等。生成式AI的突破在于其创造性和灵活性，但也面临数据隐私、版权保护等挑战。

根据AI的具体能力再细分：

① 视觉识别模型 (Computer Vision Model)

视觉识别模型让计算机能“看懂”并解析图像与视频内容，属于计算机视觉领域。主要任务包括图像分类、物体检测、图像分割等。模型能准确辨识影像中的物体、人脸、文字或特定场景。其核心在于从像素中提取特征，并与已知模式进行比对，以完成识别、定位或追踪等任务。代表模型有YOLO、ResNet，主要使用场景：

- **智能制造：** 在生产线上部署视觉识别系统，能即时检测产品外观的微小瑕疵，如刮痕或缺件，自动剔除不合格品，确保出厂品质，准确率远超人眼。
- **医疗影像分析：** 医院导入AI辅助判读系统，分析X光或CT扫描影像。模型能快速标记出疑似肿瘤或病变的区域，协助放射科医生提高诊断效率与准确性。

② 生图/生视频模型 (Text-to-Image/Video)

专门将文字描述转换为全新的图像或视频。它们能够根据用户输入的文字提示，创造出符合描述且风格多样的视觉内容。模型能融合不同概念、属性和风格，生成前所未有的原创作品。代表模型有DALL-E、Midjourney及Sora，主要使用场景：

- **产品设计：** 设计师输入“一款富有圣诞气息的手机壳，使用环保材料”，模型可快速生成多款概念图，加速产品可视化与迭代过程。
- **影视预览：** 导演利用文字生成视频模型，将剧本中的关键场景转换为动态预览片段，以便在实际拍摄前，评估镜头、光影和场景布局的可行性。

<https://www.liblib.art/>

③ 自动驾驶模型(Autonomous Driving Model)

一套复杂的AI系统，整合了视觉识别、传感器融合、决策规划等多种模型。其目标是让车辆在无需人类干预下安全行驶，是AI技术的高度整合应用。通过摄像头、激光雷达等传感器，即时感知周遭环境，识别行人、车辆与交通标志。模型会预测其他物体的动态，并规划出最佳的行驶路径与操作。主要使用场景：

- **无人配送：** 物流公司采用自动驾驶货车，在特定园区或高速公路进行货物运输。系统能自主导航、避开障碍物并遵守交通规则，实现24小时不间断的物流运作。
- **高级辅助驾驶：** 现今许多市售车辆搭载的辅助驾驶系统，能在高速公路上自动跟车、维持车道居中。这背后就是自动驾驶模型在识别车道线与前车距离，并控制方向盘与加减速。

4 大语言模型 (Large Language Model)

大语言模型是基于海量文本数据训练的深度学习模型，属于生成式AI的一种。它能理解和生成类人类的自然语言，具备强大的文本理解、摘要、翻译、问答及内容创作能力。通过上下文关联，能进行连贯且富有逻辑的对话与写作。并且通过少量示例可以进行下游任务的学习。常见模型有GPT系列、DeepSeek, Qwen等。主要使用场景：

- 智能客服：电商网站导入基于LLM的聊天机器人，能即时理解客户复杂的售后问题，提供个性化的解决方案，大幅提升服务效率与客户满意度。
- 内容创作：营销团队使用LLM，输入产品关键字和目标受众，快速生成多版本的广告文案、社交媒体帖文与博客文章，有效降低人力成本。

实时效果反馈

1. 大语言模型的英文简称为

A CVM

B ADM

C LLM

D TTI

2. 大语言模型属于

A 分析式AI

B 生成式AI

C 判别式AI

D 以上都不正确

答案

1=>C 2=>B

什么是大模型



大模型，全称“**大语言模型**”，英文“**Large Language Model**”，缩写“**LLM**”。是一种通用自然语言生成模型，它通过对大量的文本数据进行训练，以实现生成文本、回答问题、对话生成等人类语言理解和生成的能力。

特点：

- ① 数据量大
- ② 算力大
- ③ 参数量大
- ④ 具备强大的泛化能力

大模型兴起的条件

- 硬件进步：GPU、TPU等高性能计算设备的普及，使得训练大模型成为可能。
- 算法优化：Transformer 架构的提出，极大提升了模型处理长序列数据的能力。
- 数据爆炸：互联网文本、图像、视频等数据量激增，为大模型训练提供燃料

AIGC与AGI

AIGC (Artificial Intelligence Generated Content)：人工智能生成内容，指利用人工智能技术自动生成文本、图像、音频、视频等内容。

AGI(Artificial General Intelligence)：通用人工智能，指AI能够理解、学习和应用知识和技能，需要能够处理极其广泛的问题和环境，具有很高的适应性、自主性和创造性。

AIGC是当前AI技术落地的热门方向，AIGC的快速发展（如大语言模型）为AGI提供了部分技术积累（如自然语言处理能力），但AGI需突破认知、推理等更高层能力。这是AI的终极目标，目前还没有达到。

实时效果反馈

1. 大语言模型的特点是

- ☒ A 数据量大，参数量大
- ☒ B 算力大
- ☒ C 具备强大的泛化能力
- ☒ D 以上都正确

2. 大模型兴起的条件是

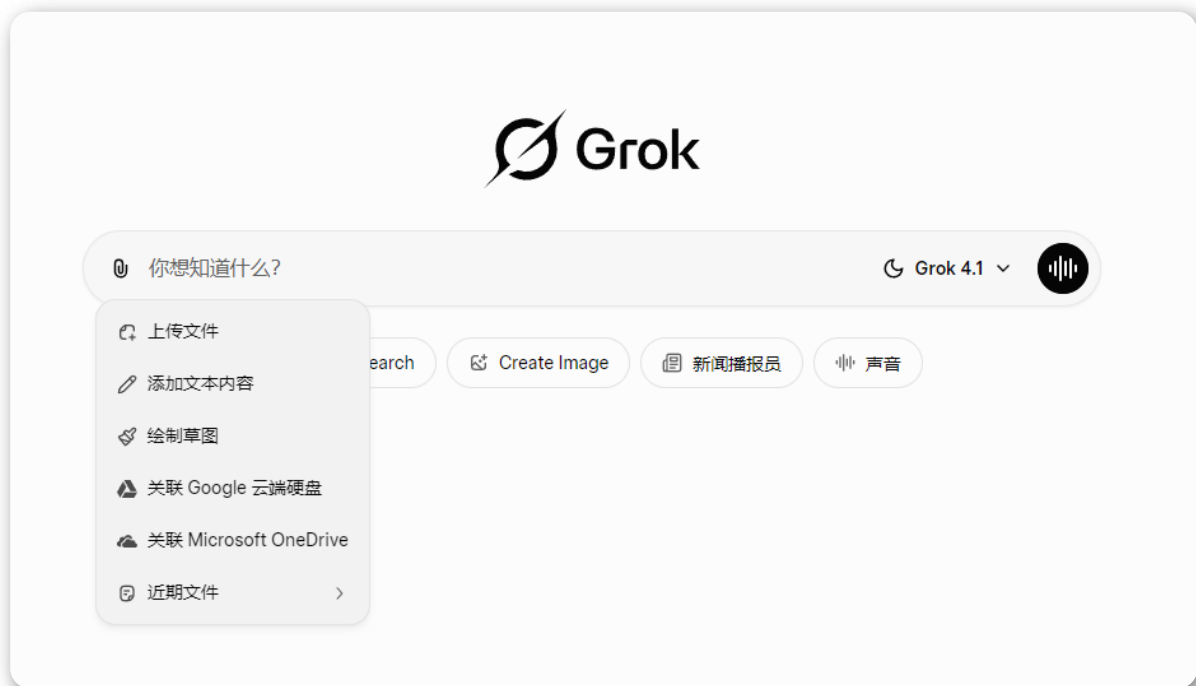
- ☒ A 硬件进步
- ☒ B 算法优化
- ☒ C 数据爆炸

D 以上都正确

答案

1=>D 2=>D

大模型聊天产品的特殊能力



联网搜索

为了弥补LLM训练数据截止日期的限制，于是AI设计了获取外部信息的能力，当用户提问涉及最新资讯时，系统会识别出这一需求，自动调用搜索功能，并将问题转化为多个简洁的搜索关键词。接着，程序调用搜索引擎（如Google搜索）获取信息。最后，这些实时信息会作为上下文提供给模型，由模型进行总结和提炼，生成精准且与时俱进的回答。

例如，当你询问“今天的股市情况怎么样”，LLM会调用一个搜索工具，输入你刚才的问题，然后获取相关的信息，整理到回答中。

读取文件

基于“检索增强生成”（Retrieval-Augmented Generation, RAG）的技术。当你上传一个文件（如PDF、Word文档）时，系统首先会将其内容分割成小块。然后，将这些文本块转化为数学向量，并存储在专门的“向量数据库”中。当你针对文件内容提问时，系统会将你的问题也转化为向量，并在数据库中快速找到最相关的文本块，最后将这些文本块连同你的问题一起交给模型，生成答案。

例如，你可以上传一份公司财报后，提问“第二季度的利润是多少？”，RAG系统能精确定位到财报中相关的片段，让LLM直接使用。

记忆功能

LLM本身是无状态的，每次对话都是一次全新的互动，不记得之前的交流。为了实现“记忆”，系统会在每次对话时，将最近的几轮问答作为背景信息一起发送给模型，该背景信息称为“短期记忆”或“上下文窗口”。

例如，你告诉AI“我喜欢简洁的回答风格”，系统会记录这一偏好。下次你提问时，它就会倾向于给出更简练的答复。

实时效果反馈

1. 大语言模型读取文件功能是基于技术的

- ☐ A AIGC
- ☐ B RAG
- ☐ C AGI
- ☐ D 以上都不正确

答案

1=>B

大模型对行业发展的影响

 保险 保险条款智能解析 文本处理效率提升30倍	 金融 金融信贷智能风控 借贷风险判断准确率提升21.5%	 医疗 医学病例自动化抽取 病例处理效率显著提升	 人力资源 候选人信息智能分类 模型识别准确率达到99%
 证券 行业新闻信息抽取 智能分析行业动态	 通信 短信内容智能分类与审核 过滤效率显著提升	 电商 电商评论观点分析 快速搭建评论数据分析系统	 物流 快递单物流地址智能识别与处理

大模型的趋势与挑战

发展趋势

1 技术突破：更大、更高效、更智能

大模型的核心能力将继续提升，但重点从“单纯更大”转向“更聪明、更省资源”。

- **模型规模的进一步扩大：**参数量继续增长（如从千亿到万亿级），以提升整体智能水平。但2025年已非盲目追求规模，而是结合MoE（Mixture of Experts，专家混合）架构，实现“总参数大、激活参数小”的高效设计（如DeepSeek、Qwen3系列）。
- **模型效率的提升：**通过量化、蒸馏、稀疏化等技术，降低推理成本和能耗。小模型（如1B-30B参数）在特定任务上接近大模型性能，开源模型（如Llama、Qwen）效率领先，推动边缘部署。
- **多模态能力的增强：**从纯文本转向整合图像、视频、音频，支持更丰富的输入输出（如图像生成、视频理解），成为主流趋势。
- **可解释性和透明性的提升：**模型决策过程更易理解，减少“黑箱”问题，提升信任和调试效率。

2 应用落地：垂直化与普惠化

大模型从实验室走向实际应用，强调定制化和大众化。

- **定制化和专用化，行业大模型爆发：**通用模型基础上细调领域专用版（如医疗、金融、法律），企业级应用爆发（如智能客服、代码生成）。
- **边缘端部署：**模型运行在手机、IoT设备上，实现低延迟、隐私保护（如移动端小模型）。
- **UGC+AIGC生态融合：**用户生成内容（UGC）与AI生成内容（AIGC）结合，形成新生态（如内容创作平台、社交AI）。

面临挑战

① 安全问题

大模型强大能力伴随风险，需要治理。

- **伦理和安全问题：**模型可能生成有害内容、偏见、虚假信息，或被恶意利用（如深假、攻击）。
- **数据和模型的治理：**训练数据隐私泄露、版权争议；模型对齐困难，确保输出符合人类价值观。
- **人机协作的优化：**AI取代部分工作，但需设计好人类监督机制，避免过度依赖或错误决策。

② 均衡问题

发展不均衡带来社会影响。

- **资源消耗和环境影响：**训练/推理需巨量电力（如H100集群），碳排放高；2025年电力成为瓶颈，推动绿色AI。
- **技术普及的不均衡：**发达国家/大厂主导，发展中国家落后；开源缓解但仍有差距，需关注公平访问和数字鸿沟。

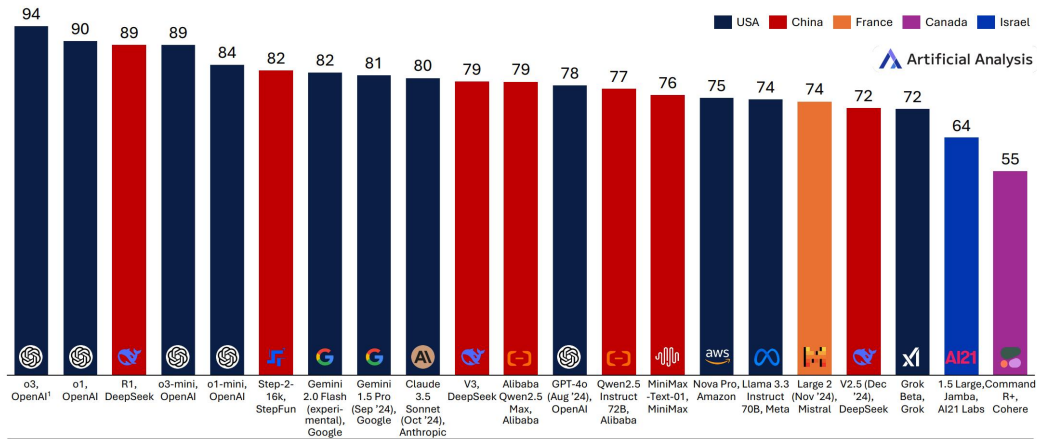
全球AI发展现状



While the US maintains an overall lead in the intelligence frontier, China is no longer far behind. Few other countries have demonstrated frontier-class training

The Language Model Frontier: Country of Origin

Artificial Analysis Intelligence Index, Selected Leading Models (Early 2025), Non-exhaustive



1. Estimated based on company claims and comparable results where available, not yet independently benchmarked by Artificial Analysis
2. A number of leading models from Chinese AI labs are excluded due to limited access or evaluation data

Artificial Analysis

- 美国：OpenAI、Anthropic、Google、Meta等公司主导前沿模型，如GPT-4o、Claude 4 Sonnet、Gemini 2.5 Flash。
- 中国：DeepSeek（如R1、V3）、阿里巴巴（如Qwen3）、Moonshot等公司快速追赶，部分模型（如Kimi K2、DeepSeek R1）已接近美国前沿水平。
- 其他地区：法国（Mistral）、加拿大（Cohere）等也有前沿模型，但中美仍是主导力量

中国模型在2025年显著缩小与美国的差距，尤其在推理模型和开源模型领域表现突出。

常见大模型介绍

模型名称	开源/闭源	发布公司	主要特点	优势领域
DeepSeek	开源	深度求索	专注于视觉与语言多模态结合，支持图像生成与推理	数学推理、编程编码、成本高效（开源MoE架构领先）
Qwen (通义千问)	开源	阿里巴巴	开源大模型，支持多语言理解与生成，具备强大推理能力	多语言处理、编码生成、长上下文理解（Qwen 3系列前沿）
GLM (智谱清言)	开源	智谱科技	聚焦中文NLP，强大跨行业能力	中文任务、工具调用、数学与复杂推理
Doubao (豆包)	闭源	字节跳动	生成式AI产品，强调多轮对话与情感理解，符合上下文文本	对话流畅性、常识推理、低成本商用部署
Hunyuan (混元)	开源	腾讯	支持多模态AI能力，处理文本、图像、视频等数据类型	多模态处理（图像/视频）、企业级应用
Kimi	开源	月之暗面	专注于自然语言处理，提供高效文本生成与理解能力	长上下文处理、编码与数学推理、多模态分析
GPT	闭源	OpenAI	强大自然语言处理能力，支持多种语言任务	通用推理、创意生成、多语言任务（o系列推理领先）
Claude	闭源	Anthropic	专注于生成安全可靠的文本，支持长文本生成	安全对齐、长文档处理、伦理与可靠响应

模型名称	开源/闭源	发布公司	主要特点	优势领域
Llama	开源	Meta	开源大模型，支持多种语言任务，注重效率与可扩展性	开源生态、自定义微调、高效部署
Gemini	闭源	Google	多模态模型，支持文本、图像和视频处理	多模态整合（图像/视频）、搜索与实时信息处理
Grok	部分开源	xAI	强调实时信息整合与工具使用，具备幽默与个性风格	实时搜索、复杂推理、创意与情感交互、工具代理

开源大模型与闭源大模型的对比：

维度	开源大模型	闭源大模型
透明度	✔ 代码、数据、训练流程公开，可审查模型行为	✗ 黑箱操作，用户无法验证内部逻辑或数据来源
定制化能力	✔ 支持微调、领域适配，灵活适应特定任务	✗ 仅限API功能，无法修改模型结构或调整底层参数
成本控制	✔ 可本地部署，长期成本固定（硬件投入为主）	✗ 按使用量付费（如Token计费），大规模应用成本高
性能上限	✗ 受限于社区资源，通常弱于头部闭源模型	✔ 依托大厂算力与数据，推理、多模态等能力更强
维护复杂性	✗ 需自行管理基础设施、安全更新和技术支持	✔ 供应商负责运维与迭代，用户无需关心底层技术细节
合规与隐私	✔ 数据完全本地化，满足严格隐私与合规要求	✗ 依赖第三方API，数据需上传外部服务器，存在泄露风险

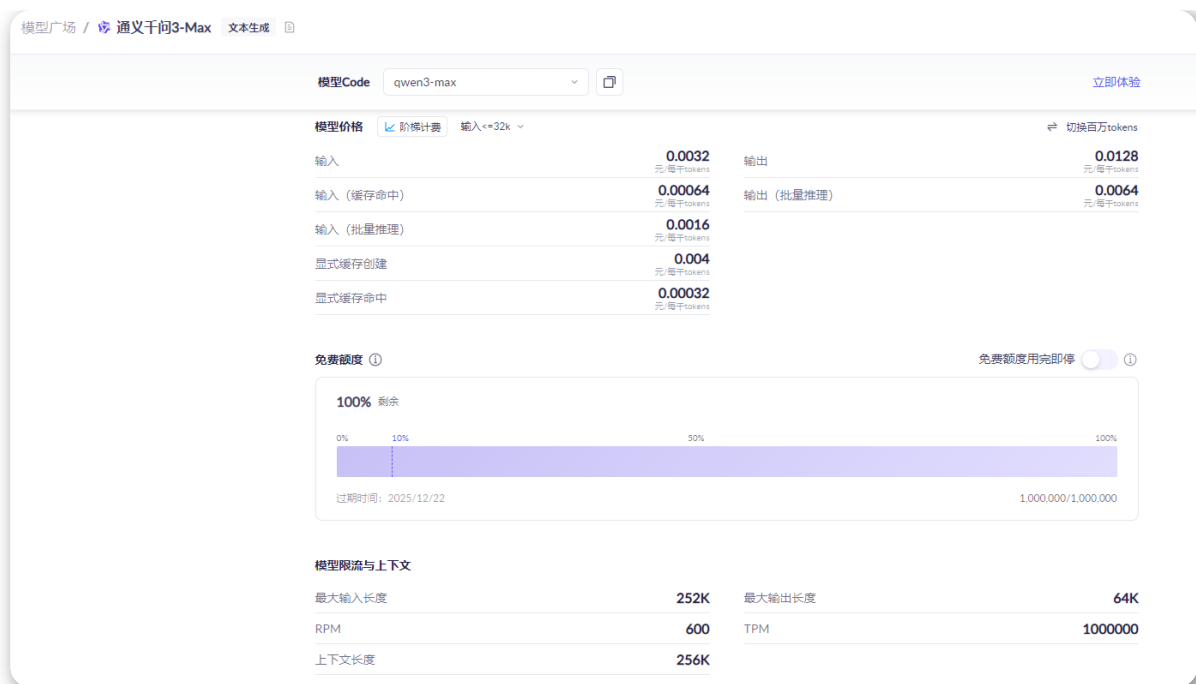
实时效果反馈

1. 以下属于开源大模型优势的是

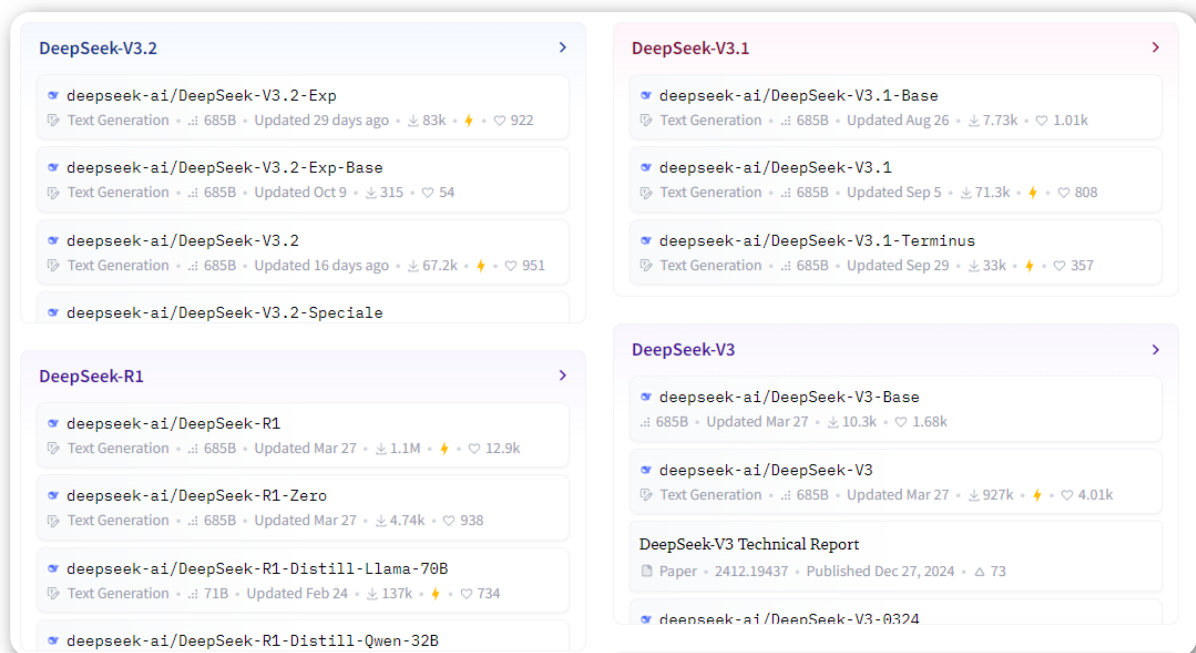
- A

支持微调、领域适配，灵活适应特定任务
- B

代码、数据、训练流程公开，可审查模型行为



- 最大输入长度：252K（约25.2万Tokens，支持超长上下文输入）
- 最大输出长度：64K（约6.4万Tokens，一次最多可生成这么多Tokens）
- RPM：600（Requests Per Minute，每分钟最大请求次数为600）
- TPM：1,000,000（Tokens Per Minute，每分钟最大处理Tokens数量为100万）
- 上下文长度：256K（总上下文窗口大小，输入+输出不超过这个值）



- DeepSeek-V3.2：DeepSeek第3.2代模型
 - Exp：实验版（Experimental），通常是预览或测试版本。
 - Base：基础版（Base Model），未经指令微调，主要用于继续预训练或特定任务微调。
 - Speciale/Terminus：特殊优化版。
 - Zero：特定训练阶段或变体。
 - Distill：蒸馏版，用更大模型蒸馏得到的小模型。

- 685B：指大模型的内部参数量，1个B为10亿。

实时效果反馈

1. 大模型上下文长度指的是

- A** 最大输入长度
- B** 最大生成长度
- C** 输入+输出的最大长度
- D** 以上都不正确

答案

1=>C

大模型硬件选择

- CPU：（Central Processing Unit，中央处理器）是计算机的核心部件，负责执行程序指令、处理数据以及协调计算机其他组件的工作。
- GPU：（Graphics Processing Unit，图形处理器）是专门用于处理图形和并行计算任务的处理器，最初设计用于加速图像渲染（如游戏、3D建模），但现在也广泛应用于科学计算、人工智能等领域。
- TPU：（Tensor Processing Unit，张量处理器）是谷歌（Google）专门为机器学习（ML）和张量运算设计的定制化加速芯片，主要用于加速神经网络的计算任务（如训练和推理）。TPU 在 AI 工作负载（如深度学习、大规模矩阵乘法）上比传统 CPU 和 GPU 更高效。

模型硬件需求对照表

模型参数	显存需求 (FP16)	内存推荐	推荐GPU配置
1.5B	2-3GB	8GB	单卡RTX 3060 (12GB)
7B	14-16GB	16GB	单卡RTX 4080 (16GB)
14B	28-32GB	32GB	单卡RTX 4090 (24GB)
32B	48-64GB	64GB	双卡RTX 4090 (24GB×2)
70B	96-128GB	128GB	4×A100 (40GB×4)
671B	808-846GB	200GB+	8×H200 (80GB×8)

实时效果反馈

1. 一张RTX 4090显卡，推荐运行多大参数量的模型

- A 14B
- B 32B
- C 70B
- D 671B

答案

1=>A

大模型应用开发入门案例



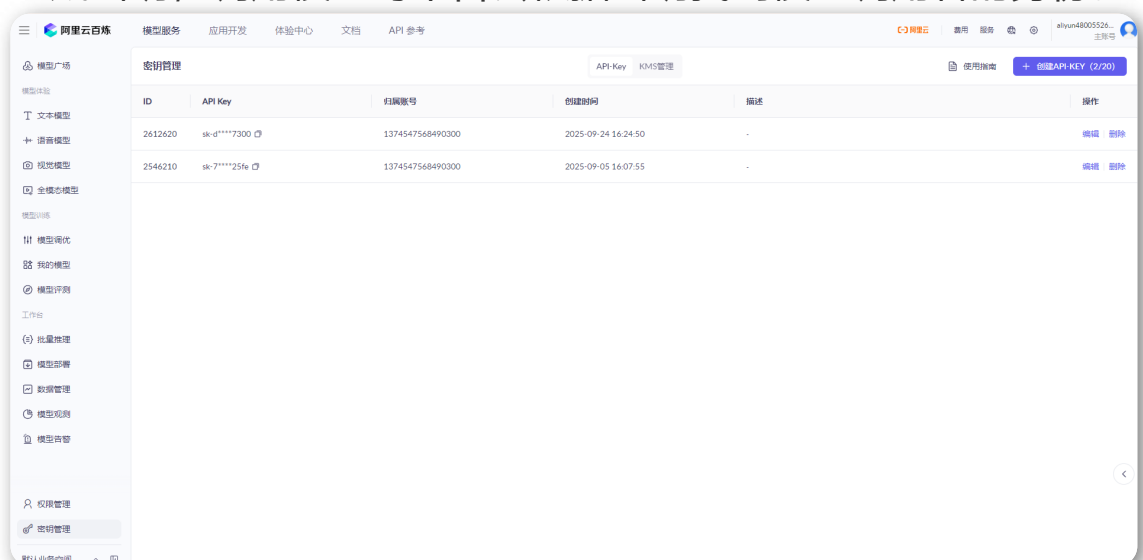
接下来我们编写一个大模型应用开发的入门案例，这里我们使用阿里云百炼平台，调用大语言模型，进行大模型应用开发。

阿里云百炼是**阿里云推出一站式大模型服务平台**，集成了国内外主流模型（如阿里通义、开源及第三方模型），提供从模型选型、应用开发到部署运维的全链路工具，降低企业使用大模型的开发门槛和成本。

- 支持阿里自研通义系列模型（如通义千问、通义视觉等），同时兼容开源模型（Llama、ChatGLM等）及第三方API。
- 用户可根据场景灵活选型，无需切换平台。
- 无需从零搭建AI基础设施，节省硬件和人力成本。

利用阿里云进行大模型应用开发的流程如下：

- 1 登录阿里云百炼<https://bailian.console.aliyun.com/#/home>
- 2 生成密钥，调用模型时平台会根据密钥找到模型调用者的身份。



- 3 打开VSCode，创建ipynb文件，选择Python3.8及以上版本的环境。

```
# 创建虚拟环境
conda create --name llm_env1 python=3.11
# 切换虚拟环境
conda activate llm_env1
```

- 4 阿里云百炼官方提供了多种编程语言的SDK，也提供了与OpenAI兼容的调用方式，因为目前国内各大模型都兼容OpenAI的API，所以我们目前也使用OpenAI兼容方式进行开发。

Python中的**OpenAI库**是官方提供的**Python SDK**，允许开发者通过代码调用OpenAI的各种AI服务，如GPT系列模型、DALL-E图像生成、Whisper语音识别等。

```
# 安装或更新OpenAI Python SDK:
pip install openai
# 如果运行失败，运行该命令
pip3 install openai
```

- 5 调用模型

```
# 导入依赖
import os
from openai import OpenAI

# 配置环境
client = OpenAI(
    # 若没有配置环境变量，请用百炼API Key将下行替换为: api_key="sk-xxx",
    api_key="sk-
d308e66c205646ddb009b3b382077300",

    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
)
```

```
# 调用模型
completion = client.chat.completions.create(
    model="qwen3-max",
    messages=[
        {"role": "system", "content": "你叫小百，是我的AI助手"},
        {"role": "user", "content": "你是谁?"},
    ],
    stream=True
)

# 输出回答
# for chunk in completion:
#     print(chunk.choices[0].delta.content,
#           end="", flush=True)

for chunk in completion:
    print(chunk)
```

实时效果反馈

1. Python使用阿里云百炼平台调用大模型，需要引入的依赖是

- ☒ A tensorflow
- ☐ B pytorch
- ☐ C openai
- ☐ D numpy

答案

1=>C

调用模型时的请求体

接下来我们介绍调用模型时请求体的写法：

```
completion = client.chat.completions.create(  
    model="qwen3-max",  
    messages=[  
        {"role": "system", "content": "You are  
a helpful assistant."},  
        {"role": "user", "content": "你是谁? "},  
    ],  
    stream=True  
)
```

- model：模型名称
- messages：传递给大模型的上下文，按对话顺序排列
 - role：角色，可以是'system' (系统指示)、'user' (用户提问) 或 'assistant' (AI回复)
 - content：该角色说的具体内容

系统提示词

- 用于设定AI的角色、行为准则和输出格式，是贯穿对话的全局指令，为其提供了扮演的“人设”。
- 应在对话开始时设定，内容要清晰明确。
- 它不应在其中包含用户的具体问题。频繁更改可能导致AI行为不稳定。

AI回复

- 模型的回复。通常用于在多轮对话中作为上下文回传给模型。
- stream：是否以流式输出方式回复。默认值为 `false`，`false`：模型生成全部内容后一次性返回；`true`：边生成边输出，每生成一部分内容即返回一个数据块（chunk）。需实时逐个读取这些块以拼接完整回复。

如果是一次性响应，获取响应的代码如下

```
print(completion.choices[0].message.content)
```

- temperature：温度参数，控制随机性，temperature越高，生成的文本更多样，反之，生成的文本更确定。取值范围：[0, 2)
- max_tokens：用于限制模型输出的最大 Token 数。若生成内容超过此值，生成将提前停止。

详细的请求参数，可以参考：<https://help.aliyun.com/zh/model-studio/qwen-api-reference>

实时效果反馈

1. 调用大模型时，参数可以控制输出的长度

- ☐ A model
- ☐ B stream
- ☐ C messages
- ☐ D max_tokens

答案

1=>D

调用模型时的响应

接下来我们介绍调用模型时响应内容的结构：

非流式输出

```
{  
  "choices": [  
    {
```

```
    "message": {
      "role": "assistant",
      "content": "我是阿里云开发的一款超大规模语言模型，我叫通义千问。",
    },
    "finish_reason": "stop",
    "index": 0,
    "logprobs": null
  }
],
"object": "chat.completion",
"usage": {
  "prompt_tokens": 3019,
  "completion_tokens": 104,
  "total_tokens": 3123
},
"created": 1735120033,
"system_fingerprint": null,
"model": "qwen-plus",
"id": "chatcmpl-6ada9ed2-7f33-9de2-8bb0-78bd4035025a"
}
```

- id: 本次调用的唯一标识符。
- choices: 模型生成内容的数组。
 - message: 模型输出的消息。
 - content: 模型的回复内容。
 - reasoning_content: 模型的思维链内容。
 - role: 消息的角色，固定为 `assistant`。
 - finish_reason: 模型停止生成的原因。有三种情况：
 - 1 触发输入参数中的 `stop` 参数，或自然停止输出时为 `stop`；
 - 2 生成长度过长而结束为 `length`；
 - 3 需要调用工具而结束为 `tool_calls`。

- index: 当前对象在 `choices` 数组中的索引。
- usage: 本次请求的 Token 消耗信息。
 - completion_tokens: 模型输出的 Token 数。
 - prompt_tokens: 输入的 Token 数。
 - total_tokens: 消耗的总 Token 数, 为 `prompt_tokens` 与 `completion_tokens` 的总和。
- created: 请求创建时的 Unix 时间戳 (秒) 。
- model: 本次请求使用的模型。

流式输出

```
[
  {
    "id": "chatcmpl-e30f5ae7-3063-93c4-90fe-beb5f900bd57",
    "choices": [
      {
        "delta": {
          "role": "assistant",
          "content": ""
        },
        "index": 0,
        "finish_reason": null
      }
    ],
    "created": 1735113344,
    "model": "qwen-plus",
    "object": "chat.completion.chunk"
  },
  {
    "id": "chatcmpl-e30f5ae7-3063-93c4-90fe-beb5f900bd57",
```

```
[{"choices": [],
  "created": 1735113344,
  "model": "qwen-plus",
  "object": "chat.completion.chunk",
  "usage": {
    "prompt_tokens": 22,
    "completion_tokens": 17,
    "total_tokens": 39
  }
}]
```

- id: 本次调用的唯一标识符。每个chunk对象有相同的 id。
- choices: 模型生成内容的数组。
 - delta: 请求的增量对象。
 - content: 增量消息内容。
 - reasoning_content: 增量的思维链内容。
 - role: 消息的角色, 固定为 `assistant`, 只在第一个chunk中有值。
 - finish_reason: 模型停止生成的原因。有四种情况:
 - 1 因触发输入参数中的 `stop` 参数, 或自然停止输出时为 `stop`;
 - 2 生成未结束时为 `null`;
 - 3 生成长度过长而结束为 `length`;
 - 4 需要调用工具而结束为 `tool_calls`。
 - index: 当前对象在 `choices` 数组中的索引。
- usage: 本次请求的 Token 消耗信息。只在最后一个chunk显示
 - completion_tokens: 模型输出的 Token 数。
 - prompt_tokens: 输入的 Token 数。
 - total_tokens: 消耗的总 Token 数, 为 `prompt_tokens` 与 `completion_tokens` 的总和。
- created: 请求创建时的 Unix 时间戳 (秒) 。
- model: 本次请求使用的模型。

详细的响应结构, 可以参考: <https://help.aliyun.com/zh/model-studio/qwen-api-reference>

实时效果反馈

1. 调用大模型时，响应的属性表示本次请求使用的模型

- A created
- B model
- C usage
- D id

答案

1=>B

使用大模型进行情感分析

接下来我们使用大模型进行一些实际的工作，我们使用大模型对用户观点评论进行情感分析，即分析一句话的情感偏向是正向还是负向的。

```
# 第一次对话
messages=[
    {"role": "system", "content": "你是一名舆情分析师，帮我判断产品口碑的正负向，回复请用一个词语：正向 或者 负向"},
    {"role": "user", "content": "这家饭店的菜品口味非常不错，推荐下次再来"}
]

completion = client.chat.completions.create(
```

```

    model="qwen3-max",
    messages=messages,
    stream=False
)
content = completion.choices[0].message.content
print(content)

# 第二次对话
messages.append({"role": "assistant",
"content": content})
messages.append( {"role": "user", "content":
"上菜慢，食材不新鲜"})

completion = client.chat.completions.create(
    model="qwen-max",
    messages=messages,
    stream=False
)
content = completion.choices[0].message.content
print(content)

```

使用大模型进行图像识别

接下来我们来使用大模型的图像识别功能：

```

completion = client.chat.completions.create(
    model="qwen3-v1-plus",
    messages=[
        {
            "role": "user",
            "content": [

```

```

        {
            "type": "image_url",
            "image_url": {
                "url":
"https://gips1.baidu.com/it/u=595975033,3712432
608&fm=3074&app=3074&f=PNG?w=2560&h=1440"
            }
        },
        {
            "type": "text",
            "text": "请详细描述这张图片的内
容，包括识别出的物体、场景和文字。"
        }
    ]
}
],
stream=True
)

# 处理模型响应结果
for chunk in completion:
    print(chunk.choices[0].delta.content,
end="", flush=True)

```

使用大模型进行图像生成

接下来我们利用大模型的图像生成功能生成图片，这里我们使用 `qwen-image-plus` 模型完成，阿里的图像生成模型的调用需要依赖 `dashscope` 包，所以我们先在虚拟环境下载 `dashscope`。

1 下载 `dashscope`


```
# 切换虚拟环境
conda activate llm_env1
# 下载dashscope
pip install dashscope
```

2 调用大模型进行图像生成

```
import json
import os
import dashscope
from dashscope import MultiModalConversation

dashscope.base_http_api_url =
'https://dashscope.aliyuncs.com/api/v1'
api_key = "sk-
ddbcf3051d3548eead43d224ea99f2b9",

messages = [
    {
        "role": "user",
        "content": [
            {"text": "一副典雅庄重的对联悬挂于厅
堂之中，房间是个安静古典的中式布置，桌子上放着一些青花
瓷，对联上左书“义本生知人机同道善思新”，右书“通云赋智
乾坤启数高志远”， 横批“智启通义”，字体飘逸，中间挂在一
着一副中国风的画作，内容是岳阳楼。"}
        ]
    }
]

response = MultiModalConversation.call(
    api_key=api_key,
    model="qwen-image-plus",
```

```

        messages=messages,
        result_format='message',          # 输出格式, 'message'表示以对话消息形式返回
        stream=False,
        watermark=False,                  # 是否添加水印
        prompt_extend=True,              # 是否启用智能提示词扩展
        negative_prompt='',              # 负面提示词, 指定不想出现的元素
        size='1328*1328'                 # 生成图像的分辨率
    )

    if response.status_code == 200:
        print(json.dumps(response,
            ensure_ascii=False))
    else:
        print(f"HTTP返回码:
{response.status_code}")
        print(f"错误码: {response.code}")
        print(f"错误信息: {response.message}")
        print("请参考文档:
https://www.alibabacloud.com/help/zh/model-studio/error-code")

```

函数功能的使用

接下来我们使用大模型的函数功能，大模型不仅可以帮助我们完成工作，还可以利用外部工具（调用函数）完成它能力之外的工作。接下来我们就让大模型通过调用天气函数，回答我们今天的天气。

大模型调用方法工具的流程是这样的：

- 1 用户向大模型提问，大模型回答。（第一轮调用）
- 2 判断大模型的回答，看是否需要调用方法，如果不需要，直接将大模型的回答响应给用户。
- 3 如果这个回答需要调用方法，则调用方法，获取方法的返回值。
- 4 将大模型的第一次回答结果和方法的返回值再次发送给大模型，大模型回答（第二轮调用），将大模型的回答响应给用户。

```
import json

# 定义获取天气的函数，实际开发时可以调用网络资源获取
def get_weather(location):
    temperate = -1
    if '北京' in location:
        temperate = 10
    elif '上海' in location:
        temperate = 30
    elif '广州' in location:
        temperate = 36

    return f"{location}当前温度是 {temperate} 摄氏度。"

# 定义函数工具，大模型会使用该工具调用函数
tools = [
    {
        "type": "function",
        "function": {
            # 方法名
            "name": "get_weather",
            # 方法描述
            "description": "获取指定地点的当前天气",
            # 方法参数
```

```

        "parameters": {
            "type": "object",
            # 参数
            "properties": {
                "location": {
                    "type": "string",
                    "description": "城市名
称，例如：北京、上海、广州"
                }
            },
            # 必填
            "required": ["location"]
        }
    }
}
]

```

```

query = "你是谁"
messages = [{"role": "user", "content": query}]

```

第一轮调用，判断模型是否需要调用工具

```

response = client.chat.completions.create(
    model="qwen3-max",
    messages=messages,
    tools=tools,
    # 让模型决定是否调用工具
    tool_choice="auto",
    stream=False
)

```

拿到模型的返回信息，信息包含模型是否需要调用函数

```

message = response.choices[0].message

```

判断是否需要调用工具

```
if message.tool_calls:
```

函数名

```
function_name =
```

```
message.tool_calls[0].function.name
```

执行本地函数

```
if function_name == "get_weather":
```

```
function_args =
```

```
json.loads(message.tool_calls[0].function.arguments)
```

```
tool_response =
```

```
get_weather(location=function_args.get("location"))
```

把模型第一轮回复加入到消息历史中

```
messages.append(message)
```

把函数执行结果加入到消息历史中

```
messages.append({
```

```
    "role": "tool",
```

```
    "tool_call_id": message.tool_calls[0].id,
```

```
    "name": function_name,
```

```
    "content": tool_response
```

```
})
```

第二轮调用，把第一轮调用的结果和函数执行结果发个模型，让它生成最终回答

```
sencod_response =
```

```
client.chat.completions.create(
```

```
    model="qwen3-max",
```

```
    messages=messages,
```

```
        tools=tools,  
        tool_choice="auto",  
        stream=False  
    )  
  
    message =  
sencod_response.choices[0].message  
  
# 输出大模型的回答  
print(message.content)
```